
1. Introduction to Linked List (Struct, Node Representation)

Problem Statement

Design a Linked List node structure and understand how a node is represented.

Pattern

👉 Core Data Structure design (Singly Linked List).

Trick to Remember

- Each node has `data + next`.
- Last node points to `NULL`.

Variation

- Singly Linked List (SLL)
- Doubly Linked List (DLL: has `prev` pointer too).

C++ Code (Singly Node)

```
struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(NULL) {}
};
```

Java Code:

```
class Node {
    int data;
    Node next;

    Node(int val) {
        this.data = val;
        this.next = null;
    }
}
```

C++ Code (Doubly Node)

```
struct Node {
    int data;
    Node* prev;
    Node* next;
    Node(int val) : data(val), prev(NULL), next(NULL) {}
};
```

```
};
```

 **Time Complexity** → $O(1)$ (just definition)

 **Space Complexity** → $O(1)$

Operations in Singly Linked List

C++ Implementation

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(NULL) {}
};

class LinkedList {
private:
    Node* head;
public:
    LinkedList() : head(NULL) {}

    // Insert at beginning
    void insertAtBeginning(int val) {
        Node* newNode = new Node(val);
        newNode->next = head;
        head = newNode;
    }

    // Insert at end
    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = newNode;
            return;
        }
        Node* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    // Search element
    bool search(int key) {
        Node* temp = head;
        while (temp) {
            if (temp->data == key) return true;
            temp = temp->next;
        }
        return false;
    }
}
```

```

// Delete element by value
void deleteNode(int key) {
    if (!head) return;

    if (head->data == key) {
        Node* toDelete = head;
        head = head->next;
        delete toDelete;
        return;
    }

    Node* temp = head;
    while (temp->next && temp->next->data != key) {
        temp = temp->next;
    }
    if (temp->next) {
        Node* toDelete = temp->next;
        temp->next = temp->next->next;
        delete toDelete;
    }
}

// Traverse / Print
void display() {
    Node* temp = head;
    while (temp) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

};

int main() {
    LinkedList list;

    list.insertAtBeginning(10);
    list.insertAtBeginning(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);

    cout << "Linked List: ";
    list.display();

    cout << "Search 30: " << (list.search(30) ? "Found" : "Not Found") <<
endl;

    list.deleteNode(20);
    cout << "After Deletion: ";
    list.display();

    return 0;
}

```

✓ Java Implementation

```
class Node {
    int data;
    Node next;

    Node(int val) {
        this.data = val;
        this.next = null;
    }
}

class LinkedList {
    private Node head;

    // Insert at beginning
    public void insertAtBeginning(int val) {
        Node newNode = new Node(val);
        newNode.next = head;
        head = newNode;
    }

    // Insert at end
    public void insertAtEnd(int val) {
        Node newNode = new Node(val);
        if (head == null) {
            head = newNode;
            return;
        }
        Node temp = head;
        while (temp.next != null) {
            temp = temp.next;
        }
        temp.next = newNode;
    }

    // Search element
    public boolean search(int key) {
        Node temp = head;
        while (temp != null) {
            if (temp.data == key) return true;
            temp = temp.next;
        }
        return false;
    }

    // Delete element by value
    public void deleteNode(int key) {
        if (head == null) return;

        if (head.data == key) {
            head = head.next;
            return;
        }

        Node temp = head;
        while (temp.next != null && temp.next.data != key) {
```

```

        temp = temp.next;
    }

    if (temp.next != null) {
        temp.next = temp.next.next;
    }
}

// Traverse / Print
public void display() {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    }
    System.out.println("NULL");
}

public static void main(String[] args) {
    LinkedList list = new LinkedList();

    list.insertAtBeginning(10);
    list.insertAtBeginning(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);

    System.out.print("Linked List: ");
    list.display();

    System.out.println("Search 30: " + (list.search(30) ? "Found" : "Not
Found"));

    list.deleteNode(20);
    System.out.print("After Deletion: ");
    list.display();
}
}

```



Find Length of Linked List (Traversal)

Problem Statement

Find the number of nodes in a Linked List.

Pattern

👉 Traversal.

Trick to Remember

- Keep `count++` while traversing until NULL.

Variation

- Iterative
- Recursive

C++ Code

```
int lengthLL(Node* head) {  
    int count = 0;  
    while (head) {  
        count++;  
        head = head->next;  
    }  
    return count;  
}
```

 **Time Complexity** → $O(N)$

 **Space Complexity** → $O(1)$

5. Search an Element in Linked List

Problem Statement

Check if a key exists in LL.

Pattern

 Traversal + Condition check.

Trick to Remember

- Traverse and compare `data == key`.

Variation

- Iterative
- Recursive

C++ Code

```
bool search(Node* head, int key) {  
    while (head) {  
        if (head->data == key) return true;  
        head = head->next;  
    }  
    return false;  
}
```

 **Time Complexity** $\rightarrow O(N)$
 **Space Complexity** $\rightarrow O(1)$

6. Doubly Linked List Basics

Overview

Operations provided:

- insertAtBeginning, insertAtEnd, insertAtPosition(pos) — (pos in 0..size0..size0..size)
 - deleteHead, deleteTail, deleteAtPosition(pos), deleteByValue(value)
 - search(value) $\rightarrow$ returns index or -1
 - displayForward, displayBackward
 - size (keeps track of number of nodes)
- Complexities:
- Insert at head/tail: $O(1)$
 - Delete head/tail: $O(1)$
 - Insert/delete/search at arbitrary position or by value: $O(n)$
-

C++ Implementation

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
    Node(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyLinkedList {
private:
    Node* head;
    Node* tail;
    int sz;

public:
    DoublyLinkedList() : head(nullptr), tail(nullptr), sz(0) {}
    ~DoublyLinkedList() {
        Node* cur = head;
        while (cur) {
            Node* nxt = cur->next;
```

```

        delete cur;
        cur = nxt;
    }
}

int size() const { return sz; }

// Insert at beginning (O(1))
void insertAtBeginning(int val) {
    Node* node = new Node(val);
    if (!head) { // empty list
        head = tail = node;
    } else {
        node->next = head;
        head->prev = node;
        head = node;
    }
    ++sz;
}

// Insert at end (O(1))
void insertAtEnd(int val) {
    Node* node = new Node(val);
    if (!tail) {
        head = tail = node;
    } else {
        tail->next = node;
        node->prev = tail;
        tail = node;
    }
    ++sz;
}

// Insert at position (0-based). valid pos: 0..sz
void insertAtPosition(int val, int pos) {
    if (pos < 0 || pos > sz) {
        cout << "Insert: Position out of range\n";
        return;
    }
    if (pos == 0) { insertAtBeginning(val); return; }
    if (pos == sz) { insertAtEnd(val); return; }

    // choose direction based on pos for efficiency
    Node* cur;
    if (pos <= sz/2) {
        cur = head;
        for (int i = 0; i < pos; ++i) cur = cur->next;
    } else {
        cur = tail;
        for (int i = sz-1; i >= pos; --i) cur = cur->prev;
    }
    Node* node = new Node(val);
    Node* prev = cur->prev;
    // splice node between prev and cur
    prev->next = node;
    node->prev = prev;
    node->next = cur;
}

```

```

        cur->prev = node;
        ++sz;
    }

    // Search returns first index or -1 (O(n))
    int search(int key) const {
        Node* cur = head;
        int idx = 0;
        while (cur) {
            if (cur->data == key) return idx;
            cur = cur->next;
            ++idx;
        }
        return -1;
    }

    // Delete head (O(1))
    void deleteHead() {
        if (!head) return;
        Node* toDelete = head;
        if (head == tail) { // single node
            head = tail = nullptr;
        } else {
            head = head->next;
            head->prev = nullptr;
        }
        delete toDelete;
        --sz;
    }

    // Delete tail (O(1))
    void deleteTail() {
        if (!tail) return;
        Node* toDelete = tail;
        if (head == tail) { // single node
            head = tail = nullptr;
        } else {
            tail = tail->prev;
            tail->next = nullptr;
        }
        delete toDelete;
        --sz;
    }

    // Delete at position (0-based) (O(n) worst)
    void deleteAtPosition(int pos) {
        if (pos < 0 || pos >= sz) {
            cout << "Delete: Position out of range\n";
            return;
        }
        if (pos == 0) { deleteHead(); return; }
        if (pos == sz-1) { deleteTail(); return; }

        Node* cur;
        if (pos <= sz/2) {
            cur = head;
            for (int i = 0; i < pos; ++i) cur = cur->next;

```

```

        } else {
            cur = tail;
            for (int i = sz-1; i > pos; --i) cur = cur->prev;
        }
        Node* p = cur->prev;
        Node* n = cur->next;
        p->next = n;
        n->prev = p;
        delete cur;
        --sz;
    }

    // Delete first occurrence by value (O(n))
    void deleteByValue(int key) {
        if (!head) return;
        Node* cur = head;
        while (cur && cur->data != key) cur = cur->next;
        if (!cur) return; // not found

        if (cur == head) { deleteHead(); return; }
        if (cur == tail) { deleteTail(); return; }

        Node* p = cur->prev;
        Node* n = cur->next;
        p->next = n;
        n->prev = p;
        delete cur;
        --sz;
    }

    // Display forward
    void displayForward() const {
        Node* cur = head;
        while (cur) {
            cout << cur->data;
            if (cur->next) cout << " <-> ";
            cur = cur->next;
        }
        cout << " -> NULL\n";
    }

    // Display backward
    void displayBackward() const {
        Node* cur = tail;
        while (cur) {
            cout << cur->data;
            if (cur->prev) cout << " <-> ";
            cur = cur->prev;
        }
        cout << " -> NULL\n";
    }
};

int main() {
    DoublyLinkedList dll;

    cout << "Insert at end: 10, 20, 30\n";

```

```

dll.insertAtEnd(10);
dll.insertAtEnd(20);
dll.insertAtEnd(30);
dll.displayForward();    // 10 <-> 20 <-> 30 -> NULL
dll.displayBackward();   // 30 <-> 20 <-> 10 -> NULL

cout << "Insert at beginning: 5\n";
dll.insertAtBeginning(5);
dll.displayForward();    // 5 <-> 10 <-> 20 <-> 30 -> NULL

cout << "Insert 15 at position 2 (0-based)\n";
dll.insertAtPosition(15, 2); // list: 5,10,15,20,30
dll.displayForward();

cout << "Search 20 -> index: " << dll.search(20) << "\n";

cout << "Delete by value 15\n";
dll.deleteByValue(15);
dll.displayForward();

cout << "Delete at position 0 (head)\n";
dll.deleteAtPosition(0);
dll.displayForward();

cout << "Delete tail\n";
dll.deleteTail();
dll.displayForward();

cout << "Final size: " << dll.size() << "\n";
return 0;
}

```

Java Implementation

```

public class DoublyLinkedList {
    private static class Node {
        int data;
        Node prev;
        Node next;
        Node(int val) { data = val; prev = null; next = null; }
    }

    private Node head;
    private Node tail;
    private int sz;

    public DoublyLinkedList() {
        head = null;
        tail = null;
        sz = 0;
    }

    public int size() { return sz; }
}

```

```

// Insert at beginning O(1)
public void insertAtBeginning(int val) {
    Node node = new Node(val);
    if (head == null) {
        head = tail = node;
    } else {
        node.next = head;
        head.prev = node;
        head = node;
    }
    sz++;
}

// Insert at end O(1)
public void insertAtEnd(int val) {
    Node node = new Node(val);
    if (tail == null) {
        head = tail = node;
    } else {
        tail.next = node;
        node.prev = tail;
        tail = node;
    }
    sz++;
}

// Insert at position (0-based). valid pos: 0..sz
public void insertAtPosition(int val, int pos) {
    if (pos < 0 || pos > sz) {
        System.out.println("Insert: Position out of range");
        return;
    }
    if (pos == 0) { insertAtBeginning(val); return; }
    if (pos == sz) { insertAtEnd(val); return; }

    Node cur;
    if (pos <= sz/2) {
        cur = head;
        for (int i = 0; i < pos; i++) cur = cur.next;
    } else {
        cur = tail;
        for (int i = sz-1; i >= pos; i--) cur = cur.prev;
    }

    Node node = new Node(val);
    Node prev = cur.prev;
    prev.next = node;
    node.prev = prev;
    node.next = cur;
    cur.prev = node;
    sz++;
}

// Search first occurrence, return index or -1
public int search(int key) {
    Node cur = head;
    int idx = 0;

```

```

        while (cur != null) {
            if (cur.data == key) return idx;
            cur = cur.next;
            idx++;
        }
        return -1;
    }

    // Delete head (O(1))
    public void deleteHead() {
        if (head == null) return;
        if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
            head.prev = null;
        }
        sz--;
    }

    // Delete tail (O(1))
    public void deleteTail() {
        if (tail == null) return;
        if (head == tail) {
            head = tail = null;
        } else {
            tail = tail.prev;
            tail.next = null;
        }
        sz--;
    }

    // Delete at position (0-based)
    public void deleteAtPosition(int pos) {
        if (pos < 0 || pos >= sz) {
            System.out.println("Delete: Position out of range");
            return;
        }
        if (pos == 0) { deleteHead(); return; }
        if (pos == sz-1) { deleteTail(); return; }

        Node cur;
        if (pos <= sz/2) {
            cur = head;
            for (int i = 0; i < pos; i++) cur = cur.next;
        } else {
            cur = tail;
            for (int i = sz-1; i > pos; i--) cur = cur.prev;
        }
        Node p = cur.prev;
        Node n = cur.next;
        p.next = n;
        n.prev = p;
        sz--;
    }

    // Delete first occurrence by value

```

```

public void deleteByValue(int key) {
    if (head == null) return;
    Node cur = head;
    while (cur != null && cur.data != key) cur = cur.next;
    if (cur == null) return;

    if (cur == head) { deleteHead(); return; }
    if (cur == tail) { deleteTail(); return; }

    Node p = cur.prev;
    Node n = cur.next;
    p.next = n;
    n.prev = p;
    sz--;
}

// Display forward
public void displayForward() {
    Node cur = head;
    while (cur != null) {
        System.out.print(cur.data);
        if (cur.next != null) System.out.print(" <-> ");
        cur = cur.next;
    }
    System.out.println(" -> NULL");
}

// Display backward
public void displayBackward() {
    Node cur = tail;
    while (cur != null) {
        System.out.print(cur.data);
        if (cur.prev != null) System.out.print(" <-> ");
        cur = cur.prev;
    }
    System.out.println(" -> NULL");
}

// Demo
public static void main(String[] args) {
    DoublyLinkedList dll = new DoublyLinkedList();

    System.out.println("Insert at end: 10,20,30");
    dll.insertAtEnd(10);
    dll.insertAtEnd(20);
    dll.insertAtEnd(30);
    dll.displayForward();
    dll.displayBackward();

    System.out.println("Insert at beginning: 5");
    dll.insertAtBeginning(5);
    dll.displayForward();

    System.out.println("Insert 15 at position 2 (0-based)");
    dll.insertAtPosition(15, 2); // 5,10,15,20,30
    dll.displayForward();
}

```

```

        System.out.println("Search 20 -> index: " + dll.search(20));

        System.out.println("Delete by value 15");
        dll.deleteByValue(15);
        dll.displayForward();

        System.out.println("Delete at position 0 (head)");
        dll.deleteAtPosition(0);
        dll.displayForward();

        System.out.println("Delete tail");
        dll.deleteTail();
        dll.displayForward();

        System.out.println("Final size: " + dll.size());
    }
}

```

Circular linked list

✓ Operations to Cover

- **Insert at Beginning**
- **Insert at End**
- **Insert at Position**
- **Search a value**
- **Delete Head**
- **Delete Tail**
- **Delete at Position**
- **Delete by Value**
- **Traverse/Display** (loops back to head safely)
- **Size()**

⚡ C++ Implementation (Singly Circular Linked List)

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

```

```

class CircularLinkedList {
private:
    Node* head;
    Node* tail;
    int sz;

public:
    CircularLinkedList() : head(nullptr), tail(nullptr), sz(0) {}
    ~CircularLinkedList() {
        while (sz > 0) deleteHead();
    }

    int size() const { return sz; }

    // Insert at beginning (O(1))
    void insertAtBeginning(int val) {
        Node* node = new Node(val);
        if (!head) { // empty
            head = tail = node;
            node->next = node; // self loop
        } else {
            node->next = head;
            head = node;
            tail->next = head; // maintain circular
        }
        sz++;
    }

    // Insert at end (O(1))
    void insertAtEnd(int val) {
        Node* node = new Node(val);
        if (!head) {
            head = tail = node;
            node->next = node;
        } else {
            tail->next = node;
            tail = node;
            tail->next = head;
        }
        sz++;
    }

    // Insert at position (0..sz)
    void insertAtPosition(int val, int pos) {
        if (pos < 0 || pos > sz) {
            cout << "Insert: Position out of range\n";
            return;
        }
        if (pos == 0) { insertAtBeginning(val); return; }
        if (pos == sz) { insertAtEnd(val); return; }

        Node* cur = head;
        for (int i = 0; i < pos-1; i++) cur = cur->next;

        Node* node = new Node(val);
        node->next = cur->next;
    }
};

```

```

        cur->next = node;
        sz++;
    }

// Search O(n)
int search(int key) const {
    if (!head) return -1;
    Node* cur = head;
    int idx = 0;
    do {
        if (cur->data == key) return idx;
        cur = cur->next;
        idx++;
    } while (cur != head);
    return -1;
}

// Delete head (O(1))
void deleteHead() {
    if (!head) return;
    if (head == tail) { // single node
        delete head;
        head = tail = nullptr;
    } else {
        Node* toDelete = head;
        head = head->next;
        tail->next = head;
        delete toDelete;
    }
    sz--;
}

// Delete tail (O(n))
void deleteTail() {
    if (!head) return;
    if (head == tail) {
        delete head;
        head = tail = nullptr;
    } else {
        Node* cur = head;
        while (cur->next != tail) cur = cur->next;
        delete tail;
        tail = cur;
        tail->next = head;
    }
    sz--;
}

// Delete at position
void deleteAtPosition(int pos) {
    if (pos < 0 || pos >= sz) {
        cout << "Delete: Position out of range\n";
        return;
    }
    if (pos == 0) { deleteHead(); return; }
    if (pos == sz-1) { deleteTail(); return; }
}

```

```

        Node* cur = head;
        for (int i = 0; i < pos-1; i++) cur = cur->next;

        Node* toDelete = cur->next;
        cur->next = toDelete->next;
        delete toDelete;
        sz--;
    }

    // Delete by value
    void deleteByValue(int key) {
        if (!head) return;
        if (head->data == key) { deleteHead(); return; }
        Node* cur = head;
        while (cur->next != head && cur->next->data != key) cur = cur->next;
        if (cur->next != head) {
            Node* toDelete = cur->next;
            cur->next = toDelete->next;
            if (toDelete == tail) tail = cur;
            delete toDelete;
            sz--;
        }
    }

    // Display forward
    void display() const {
        if (!head) { cout << "Empty\n"; return; }
        Node* cur = head;
        do {
            cout << cur->data << " -> ";
            cur = cur->next;
        } while (cur != head);
        cout << "(back to head)\n";
    }
};

int main() {
    CircularLinkedList cll;
    cll.insertAtEnd(10);
    cll.insertAtEnd(20);
    cll.insertAtEnd(30);
    cll.display();

    cll.insertAtBeginning(5);
    cll.display();

    cll.insertAtPosition(15, 2);
    cll.display();

    cout << "Search 20: index " << cll.search(20) << "\n";

    cll.deleteByValue(15);
    cll.display();

    cll.deleteHead();
    cll.display();
}

```

```

        cll.deleteTail();
        cll.display();

        cout << "Final size: " << cll.size() << "\n";
        return 0;
    }

```

Java Implementation (Singly Circular Linked List)

```

public class CircularLinkedList {
    private static class Node {
        int data;
        Node next;
        Node(int val) { data = val; next = null; }
    }

    private Node head;
    private Node tail;
    private int sz;

    public CircularLinkedList() { head = tail = null; sz = 0; }
    public int size() { return sz; }

    // Insert at beginning
    public void insertAtBeginning(int val) {
        Node node = new Node(val);
        if (head == null) {
            head = tail = node;
            node.next = node;
        } else {
            node.next = head;
            head = node;
            tail.next = head;
        }
        sz++;
    }

    // Insert at end
    public void insertAtEnd(int val) {
        Node node = new Node(val);
        if (head == null) {
            head = tail = node;
            node.next = node;
        } else {
            tail.next = node;
            tail = node;
            tail.next = head;
        }
        sz++;
    }
}

```

```

// Insert at position
public void insertAtPosition(int val, int pos) {
    if (pos < 0 || pos > sz) {
        System.out.println("Insert: Position out of range");
        return;
    }
    if (pos == 0) { insertAtBeginning(val); return; }
    if (pos == sz) { insertAtEnd(val); return; }

    Node cur = head;
    for (int i = 0; i < pos-1; i++) cur = cur.next;

    Node node = new Node(val);
    node.next = cur.next;
    cur.next = node;
    sz++;
}

// Search
public int search(int key) {
    if (head == null) return -1;
    Node cur = head;
    int idx = 0;
    do {
        if (cur.data == key) return idx;
        cur = cur.next;
        idx++;
    } while (cur != head);
    return -1;
}

// Delete head
public void deleteHead() {
    if (head == null) return;
    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        tail.next = head;
    }
    sz--;
}

// Delete tail
public void deleteTail() {
    if (head == null) return;
    if (head == tail) {
        head = tail = null;
    } else {
        Node cur = head;
        while (cur.next != tail) cur = cur.next;
        tail = cur;
        tail.next = head;
    }
    sz--;
}

```

```

// Delete at position
public void deleteAtPosition(int pos) {
    if (pos < 0 || pos >= sz) {
        System.out.println("Delete: Position out of range");
        return;
    }
    if (pos == 0) { deleteHead(); return; }
    if (pos == sz-1) { deleteTail(); return; }

    Node cur = head;
    for (int i = 0; i < pos-1; i++) cur = cur.next;

    cur.next = cur.next.next;
    sz--;
}

// Delete by value
public void deleteByValue(int key) {
    if (head == null) return;
    if (head.data == key) { deleteHead(); return; }

    Node cur = head;
    while (cur.next != head && cur.next.data != key) cur = cur.next;

    if (cur.next != head) {
        if (cur.next == tail) tail = cur;
        cur.next = cur.next.next;
        sz--;
    }
}

// Display
public void display() {
    if (head == null) { System.out.println("Empty"); return; }
    Node cur = head;
    do {
        System.out.print(cur.data + " -> ");
        cur = cur.next;
    } while (cur != head);
    System.out.println("(back to head)");
}

public static void main(String[] args) {
    CircularLinkedList cll = new CircularLinkedList();

    cll.insertAtEnd(10);
    cll.insertAtEnd(20);
    cll.insertAtEnd(30);
    cll.display();

    cll.insertAtBeginning(5);
    cll.display();

    cll.insertAtPosition(15, 2);
    cll.display();

    System.out.println("Search 20: index " + cll.search(20));
}

```

```

        cll.deleteByValue(15);
        cll.display();

        cll.deleteHead();
        cll.display();

        cll.deleteTail();
        cll.display();

        System.out.println("Final size: " + cll.size());
    }
}

```

2. Reversal Variations

2.1 Reverse a Linked List [Iterative]

Problem Statement

Given the head of a singly linked list, reverse it and return the new head.

Pattern

 Pointer Manipulation

Trick to Remember

- Use **3 pointers** $\rightarrow$ prev, curr, next.
- Update links as you go:

$$\text{next} = \text{curr} \rightarrow \text{next} \rightarrow \text{curr} \rightarrow \text{next} = \text{prev} \rightarrow \text{prev} = \text{curr} \rightarrow \text{curr} = \text{next}.$$

Variation

- Iterative method ($O(N)$, $O(1)$).

C++ Code

```

Node* reverseIterative(Node* head) {
    Node* prev = NULL, *curr = head, *next = NULL;
    while (curr) {

```

```

        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev; // new head
}

```

 Time $\rightarrow O(N)$

 Space $\rightarrow O(1)$

🔹 2.2 Reverse a Linked List [Recursive]

Problem Statement

Reverse a linked list using recursion.

Pattern

👉 Recursion (stack unwinding fixes links).

Trick to Remember

- Base case $\rightarrow$ if `head == NULL || head->next == NULL` return head.
- Recursive call reverses the rest, then fix link:
`head->next->next = head; head->next = NULL;.`

Variation

- Recursive (elegant, but uses stack $O(N)$).

C++ Code

```

Node* reverseRecursive(Node* head) {
    if (!head || !head->next) return head;
    Node* newHead = reverseRecursive(head->next);
    head->next->next = head;
    head->next = NULL;
    return newHead;
}

```

 Time $\rightarrow O(N)$

 Space $\rightarrow O(N)$ (stack recursion)

◆ 2.3 Reverse a Doubly Linked List (DLL)

Problem Statement

Reverse a doubly linked list.

Pattern

👉 Swap `prev` and `next` for each node.

Trick to Remember

- Traverse DLL, swap pointers at each step.
- Final `prev` is new head.

Variation

- Iterative swap.

C++ Code

```
Node* reverseDLL(Node* head) {
    Node* curr = head, *prevNode = NULL;
    while (curr) {
        prevNode = curr->prev;
        curr->prev = curr->next;
        curr->next = prevNode;
        curr = curr->prev; // move forward (since swapped)
    }
    return prevNode ? prevNode->prev : NULL; // new head
}
```

🕒 Time $\rightarrow O(N)$

📦 Space $\rightarrow O(1)$

◆ 2.4 Reverse LL in Group of Size K (Hard)

Problem Statement

Reverse nodes of a linked list in groups of k .

Pattern

👉 Recursion + K-group reversal.

Trick to Remember

- Reverse first k nodes normally.
- Recur for remaining list.

- Connect head->next to recursive result.

Variation

- Iterative (stack based)
- Recursive (cleaner).

C++ Code

```
//1. declare temp = head, previous Group Tail = null

//2.for every group find kth node

//3.remember the rest nodes, and set kth->next null(broke the link)

//4. just reverse the group

//5.if 1st time haed = revHead, if not prevGroupTail->next = revHead;

//6.reconnect the broken link in step 3

//7.remember the prev group tail and move temp to next group head
```

```
class Solution {
public:
    ListNode* findkthNode(ListNode* curr, int k){
        ListNode* temp = curr;

        k--;

        while(k > 0 && temp != nullptr){
            temp = temp->next;

            k--;
        }

        return temp;
    }

    ListNode* reverse(ListNode* curr){
        if(!curr || !curr->next) return curr;
```

```

    ListNode* rest = reverse(curr->next);

    curr->next->next = curr;

    curr->next = nullptr;

    return rest;
}

ListNode* reverseKGroup(ListNode* head, int k) {

    //1. declare temp = head, PrevGroupTail = null

    ListNode* curr = head;

    ListNode* groupPrev = nullptr; //remember it

    while(curr != nullptr){

        //for every group find kth node

        ListNode* kth = findkthNode(curr,k);

        if(kth == nullptr){

            break;

        }

        //remember the rest nodes, and set kth->next null

        ListNode* groupNext = kth->next;

        kth->next = nullptr; //broke the link to reverse it

        //just reverse the group

        ListNode* revHead = reverse(curr);

        //if 1st time haed = revHead, if not prevGroupTail->next =
revHead;

        if(curr == head) head = revHead;

        else groupPrev->next = revHead;

        //reconnect the broken link

```

```

        curr->next = groupNext;

        //remember the prev group tail and move temp to next group head

        groupPrev = curr;

        curr = groupNext;

    }

    return head;

}

};

```

OR

```

Node* reverseKGroup(Node* head, int k) {
    Node* curr = head, *prev = NULL, *next = NULL;
    int count = 0;
    Node* temp = head;
    for (int i = 0; i < k && temp; i++) temp = temp->next;
    if (!temp && count < k) return head; // less than k nodes, no reverse

    curr = head;
    while (curr && count < k) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
        count++;
    }
    if (next) head->next = reverseKGroup(next, k);
    return prev;
}

```



Time $\rightarrow O(N)$



Space $\rightarrow O(N/k)$ recursion stack



3. Middle & Two-Pointer Techniques

◆ 3.1 Middle of Linked List [Tortoise-Hare]

Problem Statement

Find the middle node of a linked list.

Pattern

👉 Fast and Slow Pointer

Trick to Remember

- Slow moves 1 step.
- Fast moves 2 steps.
- When fast ends, slow is middle.

Variation

- Return 1st middle (when even length).
- Return 2nd middle (when even length).

C++ Code

```
Node* middleNode(Node* head) {  
    Node* slow = head, *fast = head;  
    while (fast && fast->next) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
    return slow;  
}
```

🕒 Time → $O(N)$

📄 Space → $O(1)$

◆ 3.2 Detect a Loop in Linked List

Problem Statement

Check if a loop exists in LL.

Pattern

👉 Floyd's Cycle Detection (Tortoise-Hare).

Trick to Remember

- If $slow == fast \rightarrow$ loop detected.

Variation

- Hashing approach ($O(N)$ space).
- Floyd's Cycle ($O(1)$ space).

C++ Code

```
bool hasCycle(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

 Time $\rightarrow O(N)$

 Space $\rightarrow O(1)$

◆ 3.3 Find the Starting Point of Loop

Problem Statement

If a loop exists, find its starting node.

Pattern

 Floyd's Algorithm extension.

Trick to Remember

- After detection, put one pointer at head.
- Move both one step $\rightarrow$ meet at loop start.

Variation

- Works only after detecting cycle.

C++ Code

```
Node* detectCycle(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) {

```

```

        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}
}
return NULL;
}

```

 Time $\rightarrow O(N)$

 Space $\rightarrow O(1)$

◆ 3.4 Length of Loop in Linked List

Problem Statement

Find the length of the loop in a LL.

Pattern

 Floyd's Cycle + Count steps.

Trick to Remember

- After meeting point, move one pointer until it comes back $\rightarrow$ count steps.

Variation

- Only works if cycle exists.

C++ Code

```

int lengthOfLoop(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            int count = 1;
            Node* temp = slow->next;
            while (temp != slow) {
                count++;
                temp = temp->next;
            }
            return count;
        }
    }
    return 0;
}

```

🕒 Time $\rightarrow O(N)$
📄 Space $\rightarrow O(1)$

✅ Now you have a **complete master breakdown** of:

- Reversal variations
 - Two-pointer techniques
-

👉 Do you want me to continue the same format for **Deletion variations + Palindrome/Rearrangement patterns** next, so that you'll have the entire **Linked List Handbook** ready?

◆ Problem Recap

A number is called a **happy number** if repeatedly replacing the number with the **sum of squares of its digits** eventually leads to 1.

If it falls into a cycle that doesn't include 1, then it's not a happy number.

Example:

- $19 \rightarrow 82 \rightarrow 68 \rightarrow 100 \rightarrow 1$ ✅ $\rightarrow$ Happy number.
 - $2 \rightarrow 4 \rightarrow 16 \rightarrow 37 \rightarrow 58 \rightarrow 89 \rightarrow 145 \rightarrow 42 \rightarrow 20 \rightarrow 4$ 🔄 $\rightarrow$ Cycle, not happy.
-

◆ Floyd's Cycle Detection Idea

- Treat the "sum of squares" process as a **linked list** where each number points to the next transformation.
 - Use **slow** and **fast** pointers:
 - $slow = f(slow)$ (one step at a time)
 - $fast = f(f(fast))$ (two steps at a time)
 - If $fast == 1 \rightarrow$ it's a happy number.
 - If $slow == fast$ but not 1 $\rightarrow$ cycle detected $\rightarrow$ not happy.
-

✅ C++ Solution (Floyd's Cycle Algorithm)

```
#include <bits/stdc++.h>
using namespace std;
```

```

int getNext(int n) {
    int sum = 0;
    while (n > 0) {
        int d = n % 10;
        sum += d * d;
        n /= 10;
    }
    return sum;
}

bool isHappy(int n) {
    int slow = n;
    int fast = getNext(n);

    while (fast != 1 && slow != fast) {
        slow = getNext(slow);           // move 1 step
        fast = getNext(getNext(fast)); // move 2 steps
    }

    return fast == 1;
}

int main() {
    int n;
    cin >> n;
    if (isHappy(n))
        cout << n << " is a Happy Number\n";
    else
        cout << n << " is NOT a Happy Number\n";
}

```

Java Solution (Floyd's Cycle Algorithm)

```

class HappyNumber {
    private static int getNext(int n) {
        int sum = 0;
        while (n > 0) {
            int d = n % 10;
            sum += d * d;
            n /= 10;
        }
        return sum;
    }

    public static boolean isHappy(int n) {
        int slow = n;
        int fast = getNext(n);

        while (fast != 1 && slow != fast) {
            slow = getNext(slow);           // move 1 step
            fast = getNext(getNext(fast)); // move 2 steps
        }
    }
}

```

```

        return fast == 1;
    }

    public static void main(String[] args) {
        int n = 19;
        if (isHappy(n))
            System.out.println(n + " is a Happy Number");
        else
            System.out.println(n + " is NOT a Happy Number");
    }
}

```

◆ Complexity

- **Time:** Each number eventually falls into a cycle $\rightarrow O(\log n)$ per step (since sum of squares $\leq 81 * \text{digits count}$).
- **Space:** $O(1) \rightarrow$ no hash set needed (unlike the hash set method).

Homework:

- **Find the Duplicate Number** (LeetCode 287)
- **Circular Array Loop** (LeetCode 457)
- **Linked List Cycle** (LeetCode 141)
- **Linked List Cycle II** (LeetCode 142)

4. Deletion Variations

◆ 4.1 Delete the Middle Node of LL

Problem Statement

Given the head of a linked list, delete the middle node. If even length, usually remove the **2nd middle**.

Pattern

👉 Fast & Slow Pointer + Deletion

Trick to Remember

- Use slow/fast pointer to reach middle.
- Maintain `prev` of slow to delete it.

Variation

- Delete 1st middle (even case).
- Delete 2nd middle (even case).

C++ Code

```
Node* deleteMiddle(Node* head) {
    if (!head || !head->next) return NULL;
    Node* slow = head, *fast = head, *prev = NULL;
    while (fast && fast->next) {
        prev = slow;
        slow = slow->next;
        fast = fast->next->next;
    }
    prev->next = slow->next;
    delete slow;
    return head;
}
```

 Time $\rightarrow O(N)$

 Space $\rightarrow O(1)$

◆ 4.2 Remove Nth Node from the Back of LL

Problem Statement

Remove the N-th node from the end of the list.

Pattern

👉 Two Pointer (distance = N).

Trick to Remember

- Move `fast` N steps ahead.
- Then move `slow` and `fast` until fast ends.
- Delete `slow->next`.

Variation

- Direct length approach (2 pass).
- Optimal 1-pass (two pointers).

C++ Code

```

Node* removeNthFromEnd(Node* head, int n) {
    Node* dummy = new Node(0);
    dummy->next = head;
    Node* fast = dummy, *slow = dummy;

    for (int i = 0; i < n; i++) fast = fast->next;
    while (fast->next) {
        slow = slow->next;
        fast = fast->next;
    }
    Node* toDelete = slow->next;
    slow->next = slow->next->next;
    delete toDelete;

    return dummy->next;
}

```

 Time $\rightarrow O(N)$

 Space $\rightarrow O(1)$

◆ What is a Dummy Node?

A **dummy node** is a "fake" node we insert at the start of a linked list. It usually has a placeholder value (like -1 or 0) and points to the head of the real list.

 It is NOT part of the actual answer; it just makes implementation easier.

◆ Why do we need a Dummy Node?

The **main problem in linked list coding** is:

- Sometimes the head itself changes (like insertion at head, deletion of head, reversing, merging, partitioning).
- Without a dummy, you need **special cases**:
 - If head is null
 - If head needs to be removed
 - If head changes after operation

A dummy node ensures that **head is never special** $\rightarrow$ all nodes (including original head) can be treated the same way.

This makes code **cleaner and safer**.

◆ When to Use a Dummy Node?

You generally use a dummy node when:

1. **Insertion/Deletion can affect the head**
 - Example: Deleting duplicates in a sorted list ($1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3$)
 - If the head itself is duplicate, without dummy you must treat it specially. With dummy $\rightarrow$ simple loop.
2. **Merging or Partitioning Lists**
 - Example: Merge two sorted lists.
 - Instead of handling first-node initialization separately, you attach everything to `dummy->next`.
3. **Reversing in Groups**
 - Example: Reverse nodes in k-groups.
 - After reversing, the head changes. With dummy, you always attach reversed groups to `prev->next`.
4. **Simplifying Return Value**
 - Instead of worrying what the new head is, just return `dummy->next`.

◆ How to Know If You Need One?

👉 Ask yourself this while solving:

- "Is there a chance that the head will change?"
 - ☒ Yes $\rightarrow$ Use dummy.
 - ☐ No $\rightarrow$ You can avoid dummy.

◆ Example: Delete Nth Node from End

Without dummy:

```
if (n == length) head = head->next; // special case
else { ... }
```

With dummy:

```
ListNode* dummy = new ListNode(0, head);
ListNode* fast = dummy, *slow = dummy;
```

👉 No special case for deleting head.

✓ In short:

Use a dummy node **when head itself might change** or **when you want to avoid special cases for the first node**

◆ 4.3 Delete All Occurrences of a Key in DLL

Problem Statement

Given head of DLL, delete all nodes with value = key.

Pattern

👉 DLL Traversal + Relinking.

Trick to Remember

- Traverse and delete wherever `node->data == key`.
- Update both `prev` and `next`.

Variation

- Key occurs at head, tail, or multiple times.

C++ Code

```
void deleteAllOccurrences(Node*& head, int key) {
    Node* curr = head;
    while (curr) {
        if (curr->data == key) {
            if (curr->prev) curr->prev->next = curr->next;
            else head = curr->next;
            if (curr->next) curr->next->prev = curr->prev;
            Node* temp = curr;
            curr = curr->next;
            delete temp;
        } else {
            curr = curr->next;
        }
    }
}
```

🕒 Time → O(N)

📄 Space → O(1)

5. Palindrome & Special Rearrangements

◆ 5.1 Check if LL is Palindrome

Problem Statement

Check if a singly linked list is palindrome.

Pattern

👉 Middle + Reverse Second Half + Compare

Trick to Remember

- Find middle (slow/fast).
- Reverse second half.
- Compare first and second halves.

Variation

- Restore list after checking (optional).
- Recursive approach (uses $O(N)$ stack).

C++ Code

```
bool isPalindrome(Node* head) {
    if (!head || !head->next) return true;
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    // reverse second half
    Node* prev = NULL, *curr = slow, *next = NULL;
    while (curr) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    Node* first = head, *second = prev;
    while (second) {
        if (first->data != second->data) return false;
        first = first->next;
        second = second->next;
    }
    return true;
}
```

 Time $\rightarrow O(N)$
 Space $\rightarrow O(1)$

◆ 5.2 Segregate Odd and Even Nodes in LL

Problem Statement

Rearrange LL so that all odd index nodes appear before even index nodes (relative order preserved).

Pattern

👉 Pointer Re-linking (Two Pointers).

Trick to Remember

- Maintain two lists: odd & even.
- Merge them at the end.

Variation

- Odd/Even by position.
- Odd/Even by value (different problem).

C++ Code

```
Node* oddEvenList(Node* head) {  
    if (!head || !head->next) return head;  
    Node* odd = head, *even = head->next, *evenHead = even;  
    while (even && even->next) { //even is always ahead then odd  
        odd->next = even->next;  
        odd = odd->next;  
        even->next = odd->next;  
        even = even->next;  
    }  
    odd->next = evenHead;  
    return head;  
}
```

 Time $\rightarrow O(N)$
 Space $\rightarrow O(1)$

◆ 5.3 Rotate a Linked List

Problem Statement

Rotate LL right by k places.

Pattern

👉 Length + Circular Linking

Trick to Remember

- Find length n .
- $k = k \% n$.
- Connect tail to head (make circular).
- Break at $(n-k)$ th node.

Variation

- Rotate left vs right.

C++ Code

```
Node* rotateRight(Node* head, int k) {
    if (!head || !head->next || k == 0) return head;
    int len = 1;
    Node* tail = head;
    while (tail->next) { tail = tail->next; len++; }
    k = k % len;
    if (k == 0) return head;
    tail->next = head; // circular

    int steps = len - k;
    Node* newTail = head;
    for (int i = 1; i < steps; i++) newTail = newTail->next;
    Node* newHead = newTail->next;
    newTail->next = NULL;
    return newHead;
}
```

🕒 Time $\rightarrow O(N)$

📄 Space $\rightarrow O(1)$

6. Sorting Variations

6.1 Sort LL (Merge Sort on Linked List)

Problem:

Sort a linked list in ascending order.

Pattern:

👉 **Divide & Conquer (Merge Sort)** – because LL doesn't support random access (so QuickSort is not efficient).

Trick:

- Find **middle** (Tortoise-Hare).
- Recursively sort left & right halves.
- Merge two sorted lists.

Variations:

- Sort in **descending order** (just change comparator).
- Sort **K-sorted list** (use heap instead).

Code:

```
class Solution {
public:
    ListNode* merge(ListNode* l1, ListNode* l2) {
        if(!l1) return l2;
        if(!l2) return l1;
        if(l1->val < l2->val) {
            l1->next = merge(l1->next, l2);
            return l1;
        } else {
            l2->next = merge(l1, l2->next);
            return l2;
        }
    }

    ListNode* getMid(ListNode* head) {
        ListNode *slow = head, *fast = head->next;
        while(fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
        }
        return slow;
    }

    ListNode* sortList(ListNode* head) {
        if(!head || !head->next) return head;
        ListNode* mid = getMid(head);
        ListNode* right = mid->next;
        mid->next = nullptr;
        return merge(sortList(head), sortList(right));
    }
};
```

Complexity:

- **Time:** $O(n \log n)$

- **Space:** $O(\log n)$ (recursion stack)
-

6.2 Sort LL of 0, 1, 2 by changing links

Problem:

Given a LL of 0, 1, 2, sort it **without counting**.

Pattern:

👉 **Three separate lists (Dutch National Flag)**

Trick:

- Create zero, one, two dummy heads.
- Attach nodes to respective lists.
- Join them together.

Variations:

- Same approach works for **segregating even/odd**.

Code:

```
ListNode* sort012(ListNode* head) {
    if(!head || !head->next) return head;

    ListNode zeroD(0), oneD(0), twoD(0);
    ListNode *zero = &zeroD, *one = &oneD, *two = &twoD;

    for(ListNode* curr=head; curr; curr=curr->next) {
        if(curr->val==0) zero->next = curr, zero=zero->next;
        else if(curr->val==1) one->next = curr, one=one->next;
        else two->next = curr, two=two->next;
    }
    zero->next = (oneD.next) ? oneD.next : twoD.next;
    one->next = twoD.next;
    two->next = NULL;
    return zeroD.next;
}
```

Complexity:

- **Time:** $O(n)$
 - **Space:** $O(1)$
-

6.3 Remove duplicates from sorted DLL

Problem:

Given a **sorted DLL**, remove duplicate nodes.

Pattern:

👉 **One pass traversal** since sorted order ensures duplicates are consecutive.

Trick:

Check `curr->data == curr->next->data` → skip node.

Code:

```
void removeDuplicates(Node* head) {
    Node* curr = head;
    while(curr && curr->next) {
        if(curr->data == curr->next->data) {
            Node* del = curr->next;
            curr->next = del->next;
            if(del->next) del->next->prev = curr;
            delete del;
        } else curr = curr->next;
    }
}
```

Complexity:

- **Time:** $O(n)$
- **Space:** $O(1)$

7. Math-based Linked List Problems

7.1 Add 1 to a number represented by LL

Problem:

LL represents a number (e.g., $1 \rightarrow 9 \rightarrow 9 = 199$). Add 1.

Pattern:

👉 **Reverse, Add, Reverse back.**

Trick:

- Reverse LL → add like normal addition with carry → reverse back.

Variations:

- Add **K** (instead of 1).
- Add **large numbers** (similar to next problem).

Code:

```
ListNode* reverse(ListNode* head) {
    ListNode* prev=NULL, *curr=head;
    while(curr) {
        ListNode* nxt=curr->next;
        curr->next=prev;
        prev=curr; curr=nxt;
    }
    return prev;
}

ListNode* addOne(ListNode* head) {
    head = reverse(head);
    ListNode* curr=head;
    int carry=1;
    while(curr && carry) {
        int sum = curr->val + carry;
        curr->val = sum%10;
        carry = sum/10;
        if(!curr->next && carry)
            curr->next = new ListNode(0);
        curr = curr->next;
    }
    return reverse(head);
}
```

Complexity:

- **Time:** $O(n)$
- **Space:** $O(1)$

7.2 Add 2 numbers in LL

Problem:

Given two LL representing numbers, return their sum as LL.

Pattern:

👉 Elementary school addition with carry.

Trick:

- Traverse both simultaneously.
- Add digit-wise + carry.
- Append new node.

Variations:

- Numbers stored in **reverse order** (LeetCode #2).
- Numbers stored in **forward order** → need to reverse first.

Code:

```
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* curr = &dummy;
    int carry=0;

    while(l1 || l2 || carry) {
        int sum = carry;
        if(l1) sum+=l1->val, l1=l1->next;
        if(l2) sum+=l2->val, l2=l2->next;
        carry = sum/10;
        curr->next = new ListNode(sum%10);
        curr = curr->next;
    }
    return dummy.next;
}
```

Complexity:

- **Time:** $O(\max(m, n))$
- **Space:** $O(1)$



8. Advanced Linked List Patterns

◆ Problem 1: Find the intersection point of Y-shaped LL

Problem Statement:

Given two singly linked lists that merge into a Y-shape at some point, find the intersection node where they merge.

Pattern:

☞ Length Difference Technique / Two Pointer Trick.

Trick to Remember:

- If two runners start from different points, make them swap lists after reaching the end → they will meet at intersection.

Approach:

1. Use **two pointers**, a and b, starting from head1 and head2.
2. Move each by 1 step.
3. When a reaches null, move it to head2. When b reaches null, move it to head1.
- 4.
5. They will meet at intersection point.

Code (C++):

```
ListNode* getIntersectionNode(ListNode* headA, ListNode* headB) {
    if(!headA || !headB) return NULL;
    ListNode* a = headA;
    ListNode* b = headB;
    while(a != b) {
        a = a ? a->next : headB;
        b = b ? b->next : headA;
    }
    return a;
}
```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

✓ Variation: Find if two linked lists intersect (just return true/false).

◆ Problem 2: Find pairs with given sum in DLL

Problem Statement:

Given a **sorted doubly linked list**, find all pairs of nodes with a given target sum.

Pattern:

→ Two Pointer Technique (like in arrays).

Trick to Remember:

- Start with left=head, right=last, move inward like 2-sum on array.

Code (C++):

```
vector<pair<int,int>> findPairs(Node* head, int target) {
    vector<pair<int,int>> ans;
    Node* left = head;
```

```

Node* right = head;
while(right->next) right = right->next; // move to last node

while(left != right && right->next != left){
    int sum = left->data + right->data;
    if(sum == target){
        ans.push_back({left->data, right->data});
        left = left->next;
        right = right->prev;
    }
    else if(sum < target) left = left->next;
    else right = right->prev;
}
return ans;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

✓ Variation: Unsorted DLL → Use hashing ($O(n)$ space).

◆ Problem 3: Flattening of LL

Problem Statement:

A linked list where each node has a `next` pointer and a `bottom` pointer. Each `bottom` list is sorted. Flatten it into one sorted list.

Pattern:

→ Merge K Sorted Lists (using merge sort or heap).

Trick to Remember:

- Treat it like merging multiple sorted lists.
- Recursively flatten right side, then merge.

Code (C++):

```

Node* merge(Node* a, Node* b){
    if(!a) return b;
    if(!b) return a;
    if(a->data < b->data){
        a->bottom = merge(a->bottom, b);
        return a;
    } else {
        b->bottom = merge(a, b->bottom);
        return b;
    }
}

```

```
Node* flatten(Node* root){
    if(!root || !root->next) return root;
    root->next = flatten(root->next);
    root = merge(root, root->next);
    return root;
}
```

Time Complexity: $O(N * \log K)$, where N =total nodes, K =#lists.

Space Complexity: $O(1)$ (recursion stack).

☑ Variation: Use Min-Heap ($O(N \log K)$).

◆ Problem 4: Clone a Linked List with Random and Next Pointer

Problem Statement:

Given a linked list with `next` and `random` pointers, create a deep copy.

Pattern:

→ HashMap or In-place Interweaving Technique.

Trick to Remember:

- Insert clone node right next to original node.
- Fix random pointers.
- Separate the lists.

Code (C++):

```
Node* copyRandomList(Node* head) {
    if(!head) return NULL;

    // Step 1: Insert cloned nodes
    for(Node* curr=head; curr; curr=curr->next->next) {
        Node* copy=new Node(curr->val);
        copy->next=curr->next;
        curr->next=copy;
    }

    // Step 2: Assign random pointers
    for(Node* curr=head; curr; curr=curr->next->next)
        if(curr->random) curr->next->random = curr->random->next;

    // Step 3: Separate lists
    Node* dummy=new Node(0);
    Node* copyTail=dummy;
    for(Node* curr=head; curr; curr=curr->next) {
        Node* copy=curr->next;
        curr->next=copy->next;
        copyTail->next=copy;
    }
}
```

```

        copyTail=copy;
    }

    return dummy->next;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$ (in-place trick)

✔ Variation: Using HashMap ($O(n)$ extra space).

class work questions

Qns

Link

Singly LL implementation - (insertion, deletion, traversal, searching)

[Middle of the Linked List](#)

[Remove Nth Node From End of List \(length method, 2 pointer method \)](#)

[Reverse Linked List](#)

Doubly LL implementation - (insertion, deletion, traversal, searching)

[Intersection of Two Linked Lists](#)

[Design Browser History](#)

Detect Cycle in LL

Length of cycle in LL

Starting point of cycle in LL

home work questions

Qns

Link

[Merge Two Sorted Lists](#)

[Delete the Middle Node of a Linked List](#)

[Add Two Numbers](#)

[Flattening a Linked List](#)

<https://www.geeksforgeeks.org/problems/flattening-a-linked-list/1>

[LRU Cache](#)

[LFU Cache](#)

challenge questions

Qns

Link

[Sort List](#)

[Reverse Nodes in k-Group](#)

